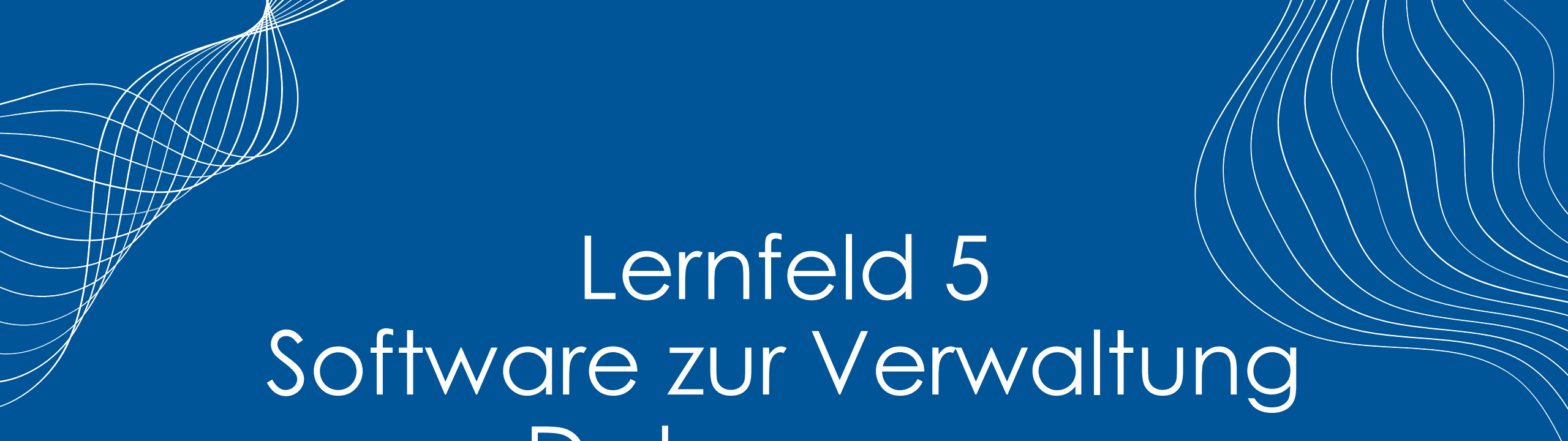




Lernfeld 5

Software zur Verwaltung von Daten anpassen

Dozent: Nicolas Kuhaupt



Einführung Programmierung - Python

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

Schleifen

Containervariablen

Funktionen

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

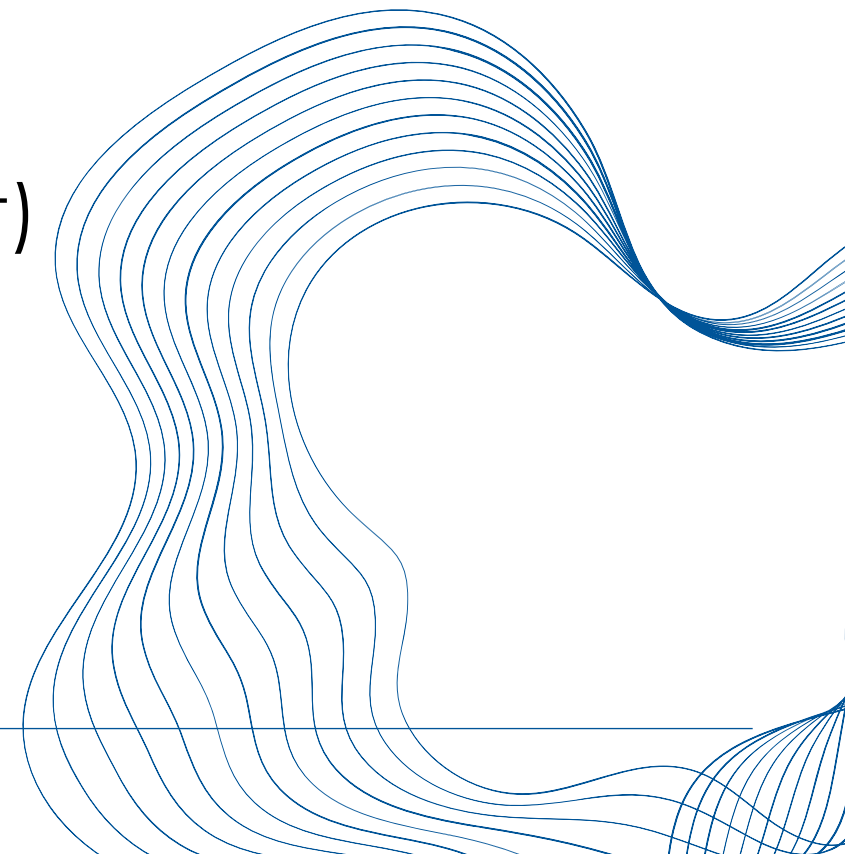
Schleifen

Containervariablen

Funktionen

Die Programmiersprache Python

- Python macht weniger Vorgaben und lässt dem Programmierer mehr Freiheiten
- Coding Standards „Python Enhancement Proposals“ (PEP) sehr verbreitet
- Nicht alles sind Klassen (Python kann Objektorientierung, erzwingt sie aber nicht)
- Programmierung in Virtuellen Umgebungen (venv)
- Einsteigerfreundlich



Aufgabe

Richte die IDE **Visual Studio Code** ein und starte das Hello-World Programm:

```
print("Hello World")
```

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

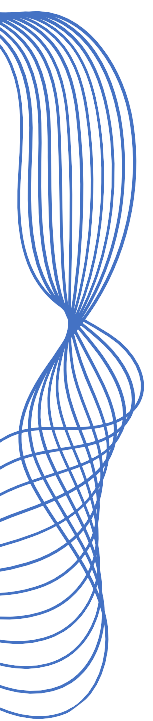
Vergleichsoperatoren und Verzweigungen mit if-else

Schleifen

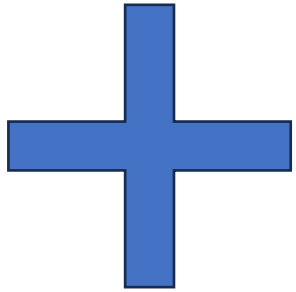
Containervariablen

Funktionen

Variablentypen

- 
- Integer
 - Float
 - String
 - Boolean

Operatoren



Aufgabe

1. Was macht der Operator „//“?
2. Was macht der Operator “%”?

3. `print(2+2*4)`

`print("2+2*4")`

Was ist der Unterschied?

4. Ein Notebook kostet 550€ ohne 19% Mehrwertsteuer. Schreibe ein Programm, das folgendes berechnet:
 - Steuerbetrag in €
 - Den Bruttopreis

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

Schleifen

Containervariablen

Funktionen

Vergleichsoperatoren

<

>

>=

<=

==



Aufgabe

Was macht der Operator **!=** ?

Verkettung von Operatoren

- Es können mehrere Bedingungen verknüpft werden durch Verkettung mit
 - Und (and)
 - Oder (or)
- Bedingungen können verneint werden / umgedreht werden
 - not



Verwendung von Entscheidungstabellen / - matrix

Aufgabe

Erstelle eine Entscheidungsmatrix für die Bedingung:
<Bedingung 1> or not <Bedingung 2>

Syntax von Verzweigungen

```
if <Bedingung>:  
    <Anweisung>  
    <Anweisung>  
else:  
    <Anweisung>
```

Verwendung von Einrückungen und Doppelpunkt ist wichtig!

Aufgabe

1. Schreibe ein Programm, das ausgibt, ob eine Zahl (int) gerade oder ungerade ist.
2. Wie kann mehr als eine Bedingung überprüft werden?
3. Schreibe ein Programm, das prüft, ob ein Jahr ein Schaltjahr ist. Ein Jahr ist ein Schaltjahr, wenn:
 - Es durch 4 teilbar ist und nicht durch 100, oder
 - Es durch 400 teilbar ist.

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

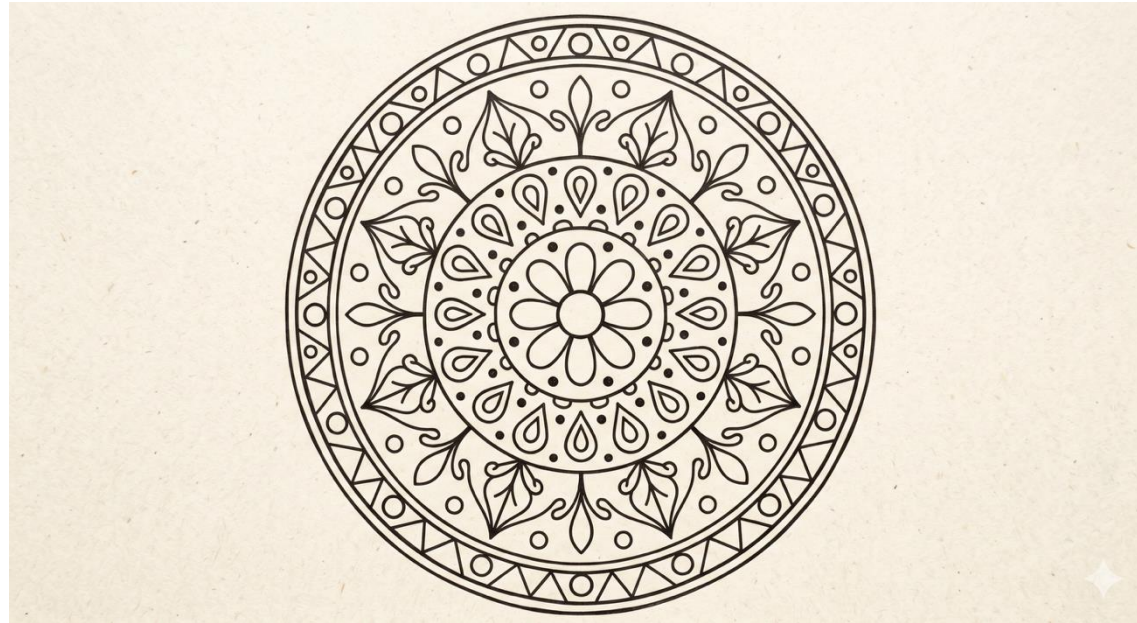
Schleifen

Containervariablen

Funktionen

Schleifen

- Mehrmaliges Ausführen einer Anweisung mit unterschiedlichen Parametern

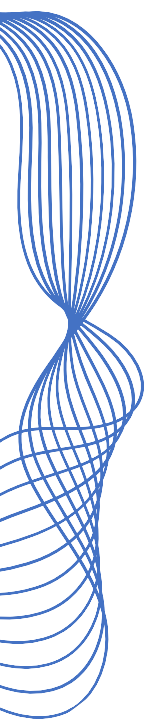


Syntax for Schleife

```
for <Parameter> in <Iterable>:  
    <Anweisung>
```

- Verwendung von Einrückung und Doppelpunkt wichtig
- Iterable im einfachsten Fall eine Liste
- Anweisung kann beliebig sein (auch erneute Schleife möglich)

Syntax while Schleife



```
while <Bedingung>:  
    <Anweisung>
```

Anweisung wird so oft ausgeführt, wie die Bedingung wahr (True) ist.

Aufgabe

1. Schreibe ein Programm, das folgende Zahlenfolgen ausgibt.
Verwende mindestens ein mal while und ein mal for:

Gerade Zahlen bis 28: 2, 4, 6, 8, 10, ..., 28

Quadrat Zahlen bis 100: 1, 4, 9, 16, 25, 36, 49, 64, 81, 100

Exponenten von 2: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512

2. Schreibe ein Programm, das die Fakultät einer Zahl berechnet (d.h. das Produkt aller Zahlen von 1 bis zur eingegebenen Zahl).

3. Schreibe ein Programm, das die ersten n Fibonacci-Zahlen berechnet und ausgibt. Verwende dafür eine for-Schleife.

Aufgabe: Passwort-Sicherheitscheck

Erstelle eine Variable `passwort = "geheim123"` (probiere später verschiedene Werte aus).

Prüfe mit `len(passwort)`, ob das Passwort mindestens 8 Zeichen lang ist.

Nutze eine for-Schleife, um jedes Zeichen im Passwort durchzugehen (`for zeichen in passwort`) und zu zählen, wie viele Ziffern (`zeichen.isdigit()`) enthalten sind.

Vergib Punkte mit Variablen: +2 für ausreichende Länge, +1 pro Ziffer.

Nutze `if/elif/else`, um am Ende auszugeben: "Schwach", "Mittel" oder "Stark" (je nach Punktzahl).

Bonus: Nutze `and/or`, um zusätzlich zu prüfen, ob mindestens ein Großbuchstabe vorkommt (`zeichen.isupper()`).



Einrichtung git

https://github.com/NKDataConv/einfuehrung_python_2026

Aufgabe Zinseszins-Rechner

Erstelle Variablen: kapital = 1000 (Startkapital), zinssatz = 0.05 (5 %), jahre = 0.

Baue eine while-Schleife, die läuft, bis sich das Kapital verdoppelt hat (kapital $\geq$ 2000).

In jeder Runde: Berechne den neuen Kontostand mit dem Operator *= (kapital *= (1 + zinssatz)), erhöhe jahre += 1.

Gib pro Durchlauf das aktuelle Jahr und den Kontostand aus.

Nutze am Ende ein if, um auszugeben, ob es weniger oder mehr als 20 Jahre gedauert hat.

Bonus: Teste mit unterschiedlichen zinssatz-Werten (2 %, 5 %, 10 %) vergleichen, wie stark sich das auf die Jahre auswirkt.

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

Schleifen

Containervariablen

Funktionen

Listen

Listen = einfache Aneinanderreihung von Variablen

- Syntax: [<Variable 1>, <Variable 2>]
- Beispiel: `my_list = [1, 2, 3]`
- Zugriff über Index
- Wichtige Befehle: `append`, `len`, `del`, `pop`, `sort`
- Iterieren mit `for`: `for <Parameter Name> in <Liste>:`

Aufgabe

1. Gegeben ist die Liste `[3,4,3,2,3,5,3,4]`. Schreibe ein Programm, das die Summe der Zahlen berechnet und ausgibt.
2. Schreibe ein Programm, das das Maximum der Liste findet und ausgibt.

Dictionaries

Dictionaries = Verknüpfung von zwei zusammengehörenden Werten

- Syntax: {<Key>: <Value>}
- Beispiel: {"cat": "Katze", "dog": "Hund"}
- Zugriff über Keys
- Wichtige Befehle: len, keys, values, items, update, del, pop
- Iterieren:
 - for <Parameter Name> in <Dictionary>.keys():
 - for <Parameter Name> in <Dictionary>.values():
 - for <Parameter Name 1>, <Parameter Name 1> in <Dictionary>.items():

Aufgabe

1. Erstelle ein Dictionary mit drei ausgedachten Namen als Key und dem Alter als Value.
2. Nun hatten alle drei Personen aus Aufgabe 1 Geburtstag. Schreibe eine Schleife, die alle Values „Alter“ im Dictionary um eins erhöht.
3. Schreibe ein Programm, das die Namen aller Personen, die nicht volljährig sind (unter 18 Jahren) in einer Liste speichert.

Agenda

Einführung und Einrichtung Python

Variablen und Operatoren

Vergleichsoperatoren und Verzweigungen mit if-else

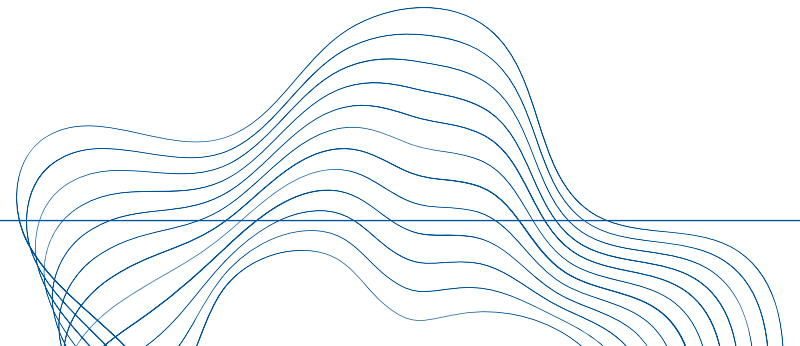
Schleifen

Containervariablen

Funktionen

Funktionen

- Erstellen von Anweisungen, die sich immer wieder (mit unterschiedlichen Parametern) aufrufen lassen
 - Vermeidung von Redundanzen
- Funktionen helfen das Programm zu strukturieren
- Funktionen können auch andere Funktionen aufrufen und beliebige Anweisungen enthalten



Funktionen Syntax

```
def <Funktionsname>(<Input Parameter>):  
    <Anweisung>  
    return <Output Parameter>
```

Wieder sind Einrückungen und Doppelpunkt essenziell für die Funktionalität.

Es kann beliebig viele (oder keine) Input-Parameter geben.

Aufgabe

1. Schreibe eine Funktion, die drei Zahlen als Input Parameter erhält und das Produkt der drei Zahlen zurückgibt.
2. Schreibe eine Funktion, die eine Liste von Zahlen als Input Parameter erhält und das Maximum und Minimum der Liste in einem Dictionary zurückgibt.
3. Schreibe eine Funktion, die als Parameter „Vermögen“, „Zinssatz“ und „Jahre“ besitzt und das Vermögen nach der Anzahl an Jahren mit dem Zinssatz zurückgibt. Teste alle deine Berechnungen. Formel:

$$\text{Vermögen} = \text{Vermögen}_{\text{Anfang}} * (1 + \text{Zinssatz})^{\text{Jahre}}$$

Aufgabe

Schreibe eine Funktion `zaehle_woerter(text)`, die einen String `text` als Parameter erhält.

Die Funktion soll die Wörter im Text zählen und ein Dictionary zurückgeben, bei dem die Keys die Wörter (kleingeschrieben) und die Werte / Values die Häufigkeit jedes Wortes sind.

Du kannst den Text z.B. am Leerzeichen aufteilen (`text.split()`), um eine Liste von Wörtern zu erhalten. Ignoriere Satzzeichen für diese Übung oder entferne sie vorher grob. Konvertiere jedes Wort in Kleinbuchstaben (`wort.lower()`), um Groß- und Kleinschreibung zu ignorieren.

Aufgabe: Drachenkampf-Simulator

Sie kämpfen gegen einen Drachen! Baue ein kleines Kampfsystem.

Erstelle Variablen für Spieler-HP, Drachen-HP und Angriffsschaden:

spieler_hp = 100, drachen_hp = 80, spieler_angriff = 15, drachen_angriff = 10

Funktion angriff_ausfuehren: Schreibe eine Funktion `angriff_ausfuehren(hp, schaden)`, die den schaden von hp abzieht und den neuen HP-Wert **zurückgibt** (return hp - schaden).

Funktion ist_besiegt: Schreibe eine Funktion `ist_besiegt(hp)`, die True zurückgibt, wenn $hp \leq 0$ ist, sonst False.

Kampfschleife: Baue eine while-Schleife, die läuft, solange weder Spieler noch Drache besiegt sind (nutze `not ist_besiegt(...)` mit and). In jeder Runde:

Rufe `angriff_ausfuehren()` auf, um drachen_hp zu verringern.

Falls der Drache danach noch nicht besiegt ist, rufe die Funktion erneut auf, um spieler_hp zu verringern.

Gib nach jeder Runde die aktuellen HP-Werte beider Kämpfer aus.

Ergebnis: Nutze nach der Schleife if/elif/else, um auszugeben, wer gewonnen hat.

Bonus – Rundenzähler mit Modulo: Führe eine Variable `runde = 1` mit, die pro Durchlauf um 1 erhöht wird. Nutze `runde % 3 == 0`, damit der Spieler alle 3 Runden einen Extra-Angriff (`spieler_angriff * 2`) ausführen darf.

Bonus – Kampflog als Liste: Führe eine Liste `kampflog = []` mit. Nach jeder Runde: `kampflog.append(f'Runde {runde}: Drache hat {drachen_hp} HP, Spieler hat {spieler_hp} HP')`. Gib am Ende die komplette Liste aus.